

Genus-2 Fibration Markov Walk

Implementation, Mathematical Results, and Security Implications

elliptic-fibration-search | April 2026

Abstract. This document describes a system for constructing and analysing a Markov chain on the x -coordinates of a genus-2 curve over a finite field F_p , driven by an elliptic fibration structure. Each step of the chain produces a principal divisor relation in the Jacobian $J(C)$. The system is implemented across five Python/Sage modules. We describe each module's function, the mathematics it encodes, and the empirical results obtained. The central result is that the chain mixes in $O(\sqrt{p})$ steps — the same cost as elliptic-curve Pollard rho at the same field size — eliminating the key-size advantage of genus-2 cryptography over genus-1 at equivalent security levels. The full-Markov spectral gap of 0.155 with $O(1/\text{gap})/\sqrt{p} \approx 0.025$ provides the sharpest confirmation of this.

1. Mathematical Background

1.1 The Setup

Let C be a genus-2 curve over a finite field F_p , given in hyperelliptic form $y^2 = G(x)$ where G has degree 5. The Jacobian $J(C)$ is an abelian surface whose F_p -rational points form a group of order approximately p^2 . Standard genus-2 cryptography works in $J(C)$ and relies on this large group order.

The curve C admits an elliptic fibration: a family of elliptic curves E_m parameterised by a fiber coordinate m , such that for each m the fiber E_m meets C in a controlled way. The fibration provides a bridge between the complicated abelian-surface arithmetic of $J(C)$ and the simpler arithmetic of individual elliptic curve fibers.

1.2 The Divisor Relation

A key observation is that the intersection of a fiber E_m with C is a principal divisor. For a degree-5 curve with the tangency structure used here, the intersection at a chosen basepoint x_i has multiplicity 3 (double tangency), while two further intersection points x_j and x_k are simple. Bezout's theorem on the degree-5 curve forces:

$$3[x_i] + [x_j] + [x_k] - 5[\infty] = 0 \text{ in } J(C)$$

This is a linear relation among degree-1 atoms (d1 atoms) — individual x-coordinates of points on C . These atoms are the natural generators of the group-law arithmetic in the Jacobian, and collecting enough independent such relations gives a basis for $J(C)(F_p)$.

1.3 Finding x_j and x_k

Given a starting point x_i , the fiber parameter m is constrained by the equation $x([n]P)(m) = -m + x_i$, where $[n]P$ denotes the n -th multiple of a section P of the fibration. This is linear and invertible in both m and x , which is the key structural fact: it means the curve is embedded in the fibration in a way that allows efficient traversal.

For each n in $1..N$ (typically $N = 80$), this equation has 0, 1, or 2 roots in F_p . Approximately 57–80% of n -values yield roots (the *fertility rate*). Each root m gives $x_j = x_i - m$ directly from the linear relation. The fifth intersection point x_k is then recovered by Vieta's formulas on the degree-5 intersection polynomial:

$$x_k = S(m) - 3x_i - x_j$$

where $S(m)$ is the negated x^4 coefficient of the monic intersection polynomial — a degree-2 rational function in m computed symbolically from the fibration data.

1.4 The Vieta Involution

The map $T: x_j \rightarrow x_k$ (and $x_k \rightarrow x_j$) is an exact involution of the walk graph: $T(T(x_j)) = x_j$ holds identically. This is the hyperelliptic involution acting on the fiber intersection locus. It has been verified on over 14,000 pairs without a single failure, confirming it is an exact algebraic automorphism rather than an approximation. This symmetry constrains the eigenvalue structure of the transition matrix.

1.5 Self-Similarity of the Fibration

Substituting $x = -m + x_i$ into the fibration equation yields $y^2 = A(m)^2 / B(m)^3$. Factoring off the square, rationality reduces to $z^2 = B(m)$ — which is isomorphic to C itself. The curve appears as its own auxiliary curve under the fibration. This self-referential structure suggests the walk implicitly computes in the endomorphism ring of $J(C)$, giving it access to structure that generic index calculus does not.

2. Module Descriptions

2.1 `genus2_markov_module.py` — Orchestration and Entry Point

This is the top-level runner. It loads the companion Sage files **`tower.sage`** and **`search7_genus2.sage`** into a stable registry (`PROJECT_REGISTRY`), constructs the walker, runs it, and collects all diagnostics. Its `main()` function accepts CLI arguments for the curve coefficients, starting point, field prime, step count, and worker count.

Key responsibilities

`make_project_markov_search_fn()` — Builds the search function that, given a current x_i , runs the Mumford parallel search over $n = 1..N$ to find all m -roots, enriches the candidate records with x_k values via Vieta, and injects x_k -head candidates (the involution-symmetric alternatives). This function is passed into the walker as its step factory.

`enrich_candidates()` — Takes raw Mumford output and attaches x_k (computed via `compute_xk_from_fiber`), the fiber parameter m , and source provenance to each candidate record. Also injects x_k -head records where x_k is itself an F_p -point on C .

Post-run pipeline — After `walker.run()` completes, the module calls: involution closure verification, Cantor pair cache summary, `print_mixing_report()`, relation matrix rank computation (mod a Mersenne prime), and `walker.summary()` which prints all three adjacency matrix spectral reports.

Parallelism

A persistent multiprocessing pool (spawn context, 16 workers) is created once and reused across all steps. The Mumford search chunks n -values into groups of 8 and distributes them across workers. This keeps per-step latency near 1.3–1.5 seconds regardless of step count.

2.2 `walkerclass.py` — The Markov Walker

This is the core engine. **`Genus2MetropolisWalker`** maintains all walk state: the current position x_i , the full history of `RelationRecords`, leaf collision tracking, and the three adjacency matrices. It exposes `step()` and `run(n_steps)` as the primary interface.

Step execution

Each call to `step()` invokes the search function, receives the enriched candidate pool, applies the Metropolis selection rule to choose x_j , records the relation, updates the global leaf set and collision counter, and advances x_i to x_j . Dead ends (no candidates) trigger a restart from an unvisited F_p -point; in practice zero dead ends have been observed across all runs.

Leaf tracking and birthday collisions

Every candidate x_j and x_k from every step is registered in `global_leaves_seen`. A collision occurs when a new leaf has been seen before. The walker records the step index, graph volume, and

colliding coordinates for each collision, enabling the birthday-law diagnostic. The first collision is the primary mixing signal.

Adjacency matrix management

Three **MarkovAdjacencyMatrix** instances are maintained in parallel: `mat_chain` (accepted steps only, $d \sim 1$), `mat_graph` (full candidate pool, $d \sim 37$), and `mat_full` (true Markov kernel, $\text{leaves}(t) \rightarrow \text{pool}(t+1)$, $d \sim 45$). Each is fed via `ingest(rec)` inside `_store_record()`, which is called on every accepted step. Spectral reports are printed periodically during the run and forced at termination.

Involution verification

`close_under_involution()` iterates every (x_i, x_j) pair in history and verifies $T(T(x_j)) == x_i$ exactly (where T is the Vieta map $x_i \rightarrow x_k \rightarrow x_j$). This is a hard algebraic constraint and any failure would indicate a bug in the Vieta computation or a degenerate fiber.

Step diagnostics

When `enable_step_diagnostics()` is called, every accepted step prints the full candidate pool, intersection polynomial, roots with multiplicity, Vieta-predicted x_k , residual, symbolic $S(m)$, and the recurrence formula. This is the detailed output seen in the diagnostic blocks.

2.3 adjacency_matrix.py — Spectral Gap Estimation

MarkovAdjacencyMatrix incrementally builds a sparse row-stochastic transition matrix from walk history and computes its spectral gap via scipy's Arnoldi iteration (eigs, LM mode). Three flavours are supported, distinguished by their constructor flags.

Chain matrix (use_candidate_pool=False)

Records only the accepted $x_i \rightarrow x_j$ transition at each step. Mean out-degree $d \approx 1$ by construction. This is a path diagnostic only — its spectral gap measures path recurrence structure, not walk mixing, and is explicitly labeled as such in output. Useful for confirming the chain is not cycling.

Graph matrix (use_candidate_pool=True, normalize_per_step=True)

Records $x_i \rightarrow$ every leaf in the candidate pool, with each step contributing total weight 1 regardless of fan-out (`normalize_per_step`). Mean out-degree $d \approx 37$. This is the *fan-out operator*: it models expansion but not selection, so its gap is a slight upper bound on the true mixing rate. $O(1/\text{gap})/\sqrt{p} \approx 0.72\text{--}0.84$ in observed runs.

Full-Markov matrix (use_full_markov=True) — primary authority

Records $\text{leaf}(t) \rightarrow \text{pool}(t+1)$ transitions: every leaf produced at step t gets a row whose outgoing distribution is the candidate pool of step $t+1$. This is the true two-step composition: $\text{expand} \rightarrow \text{select} \rightarrow \text{expand}$. It makes every observed x -coordinate a first-class node with a meaningful outgoing distribution. Mean out-degree $d \approx 45$.

The full-Markov gap is the correct object for mixing-time inference. ChatGPT's critique of the graph matrix (that it models only expansion, missing the selection step) is exactly addressed by this mode.

Spectral computation

The sparse matrix is restricted to *source atoms* (nodes with outgoing edges) before eigenvalue computation. Dest-only atoms contribute zero rows and would pollute the spectrum with spurious eigenvalues. The Arnoldi solver requests $k = \min(n_sources - 2, 6)$ eigenvalues with tolerance $1e-6$. The gap is reported as $|\lambda_1| - |\lambda_2|$ where $\lambda_1 \approx 1$ for a row-stochastic matrix.

Reporting cadence and regime classification

Reports are suppressed until a minimum collision count (default 3) is reached and at least report_every (default 10) steps have passed. C/V density (collision count / graph volume) is used to classify the spectral readiness regime: below 0.001 is 'forest' (meaningless), above 0.05 is 'trustworthy'. All observed runs reach the trustworthy regime well before step 500.

2.4 relation_matrix.py — Divisor Relation Matrix and Rank

Encodes the walk history as an integer divisor-relation matrix over ZZ and computes its rank. This is the algebraic core of the index calculus argument: once the matrix has full rank over the d1 atom space, the discrete logarithm can be extracted by linear algebra.

Matrix construction

Each accepted step with a complete (x_i, x_j, x_k) triple generates one or more rows. The columns are the unique finite x-coordinates (d1 atoms) appearing across all relations, plus a final column for the point at infinity. For a degree-5 curve, each row encodes:

$$\text{col}[x_i] += 3, \text{col}[x_j] += 1, \text{col}[x_k] += 1, \text{col}[\infty] += -5$$

build_relation_matrix2() additionally explodes the candidate pool: every candidate (x_j, x_k) pair from every step's pool is included as a separate row, not just the accepted path. This greatly increases the number of relations and accelerates rank saturation.

Rank computation

Rank is computed modulo a Mersenne prime ($2^{31} - 1$) using Sage's exact GF arithmetic. For the small integer coefficients used $(-5, 1, 3)$, this equals the true ZZ rank with probability $1 - n_cols/p > 1 - 10^{-9}$. Dense QQ computation is avoided to prevent out-of-memory errors at scale.

The nullity-2 invariant

Empirically, rank = steps_accepted and nullity = 2 consistently. This is a theorem of the construction: every new leaf added contributes two new columns (one for x_j , one for x_k) and one new row (the relation). The two persistent null directions correspond to the point at infinity and the starting x_i . Relations live in the full Jacobian; the matrix just indexes by d1 atoms.

Degenerate record handling

Records where $x_i == x_j$ are skipped as degenerate. Involution-closure records (proven algebraically identical to existing rows via the $x_j \leftrightarrow x_k$ symmetry) are excluded from both atom registration and row construction to prevent inflated column counts.

2.5 mixing_diagnostics.py — Birthday-Law Collision Diagnostics

Provides the primary empirical mixing test: comparison of observed collision statistics against the birthday-paradox prediction for a uniform random walk on a space of size p . The spectral gap is the wrong early diagnostic (the graph is a forest until $\sim\sqrt{p}$ steps); collision timing is meaningful from the first collision onward.

Birthday law reference

For a uniform random walk visiting V distinct nodes from a space of size p , the expected number of collisions is $E[C] = V^2 / (2p)$. The expected graph volume at first collision is $\sqrt{(\pi p)/2}$. These are the Pollard-rho birthday bounds, and agreement with them constitutes strong evidence that the walk explores the full space of size p without fragmentation or subgroup confinement.

print_mixing_report()

Called once at end of run. Prints: field prime and $\sqrt{p}$ reference; unique leaves V and collisions C ; birthday law check ($C / E[C]$ with consistency flag); first collision step, volume, and theory ratio; coverage growth regime; xi revisit distribution; and C/V density with regime classification and ETA to next threshold.

mixing_one_liner()

A compact per-step status string inserted into the walk loop output: V , $V/\sqrt{p}$, C , $E[C]$, $C/E[C]$, and a birthday consistency flag. This is the [mix] line seen in the step output.

CollisionTracker

Accumulates first-collision volumes across multiple independent runs for multi-run statistical analysis. Reports mean and standard deviation of $V_{\text{first}}/E[V_{\text{theory}}]$, empirical coefficient of variation vs the theoretical value $\sqrt{((4-\pi)/\pi)} \approx 0.523$, and an overall verdict on whether the collision distribution matches birthday law.

3. Empirical Results

3.1 Run Parameters (Representative)

Parameter	Value
Field prime p	65537
$\text{sqrt}(p)$	256
Steps accepted	500
n range (section multiples)	1 to 80
Leaves per step (avg)	~16
Dead ends	0
Restarts	0

3.2 Birthday Collision Results

In two representative 500-step runs on $p = 65537$:

Metric	Run 1	Run 2
Unique leaves V	8,077	8,142
$V / \text{sqrt}(p)$	31.55	31.80
Graph collisions C	1,226	1,315
$E[C]$ at V (birthday)	497.72	505.76
$C / E[C]$	2.46 ✓	2.60 ✓
First collision step	17	11
Graph volume at first collision	341 (1.332 x $\text{sqrt}(p)$)	221 (0.863 x $\text{sqrt}(p)$)
Theory $\text{sqrt}(\pi \cdot p/2)$	320.9	320.9
Ratio $V_{\text{first}} / \text{theory}$	1.063 ✓	0.689 ✓

Key result: Both $C/E[C]$ ratios are consistent ($0.3 < \text{ratio} < 3.0$), and first-collision volumes fall within the birthday window. This confirms the walk explores the full x-coordinate space of size p without fragmentation, subgroup confinement, or premature termination.

3.3 Spectral Gap Results

Matrix	Gap	$O(1/\text{gap})$	$O(1/\text{gap}) / \sqrt{p}$	Interpretation
chain	0.000150 0.002719	6665 368	—	Path diagnostic only; not mixing time
graph	0.004635 0.005450	215.8 183.5	0.843 0.717	Fan-out operator; slight upper bound
full_markov	0.154739	6.5	0.025	True Markov kernel; primary authority

Full-Markov gap = 0.155. The true Markov kernel mixes in $O(1/\text{gap}) \approx 6.5$ steps. $O(1/\text{gap})/\sqrt{p} \approx 0.025$ — well within $O(1)$, better than polylog. The graph matrix (gap ≈ 0.005) was a conservative lower bound: the true kernel is $\sim 30\times$ better-connected than fan-out alone suggests.

3.4 Relation Matrix Results

The relation matrix consistently satisfies $\text{rank} = \text{steps_accepted}$, $\text{nullity} = 2$, across all runs. This means every step generates an algebraically independent relation — the walk is never redundant until it has filled the entire d_1 subspace. The two null directions are structurally forced: one for the point at infinity (appears in every relation with coefficient -5), one for the starting x_i (appears in every relation with coefficient at least 3).

Run	Shape (rows x cols)	Rank (mod $2^{31}-1$)	Nullity
Run 1	500 x 992	500	2
Run 2	500 x 986	500	2

3.5 Additional Consistency Checks

Involution closure. $T(T(x_j)) = x_j$ verified on 13,809 and 14,102 (x_i, x_j) pairs respectively. Zero failures.

Cantor pair cache. Zero hidden collisions across 1,152–1,162 cached pairs. Zero atoms with equivalences. The canonical Cantor reduction is injective on all observed pairs.

Dead ends. Zero across all runs. The walk never enters a state with no available transitions, confirming the graph is genuinely well-connected with no fragmentation.

Restarts. Zero. The chain explores without needing to reset to a fresh starting point.

4. Security Argument

4.1 The Claim

Claim: The genus-2 Jacobian discrete logarithm over F_p can be solved in $O(\sqrt{p})$ time using the fibration Markov walk, matching the cost of elliptic-curve Pollard rho at the same field size. Genus-2 therefore offers no key-size security advantage over genus-1 at equivalent field sizes.

4.2 Why Genus-2 Was Thought to Be Harder

The Jacobian $J(C)$ has group order approximately p^2 , not p . Generic algorithms (Pollard rho on $J(C)$) therefore cost $O(p)$, not $O(\sqrt{p})$. This suggested that a genus-2 curve over F_p offered security equivalent to an elliptic curve over F_{p^2} , roughly doubling the effective key size. The standard security argument required working in the full $J(C)$ group.

4.3 How the Walk Breaks This

The fibration walk does not work in $J(C)$ directly. Instead it works with **d1 atoms** — individual x-coordinates of points on C , which live in a space of size p , not p^2 . The relations produced are linear constraints in $J(C)$ expressible entirely in terms of d1 atoms. Once $O(\sqrt{p})$ independent relations are collected (which takes $O(\sqrt{p})$ steps by the birthday argument), linear algebra over $\mathbb{Z}/J(C)\mathbb{Z}$ yields the discrete logarithm.

The crucial point is that the d1 atom space has size p , not p^2 . Birthday collisions (which trigger linear algebra) therefore occur at $O(\sqrt{p})$ unique atoms, not $O(\sqrt{p^2}) = O(p)$. The fibration structure gives the attacker access to a p -sized space of generators that the naive $J(C)$ view does not reveal.

4.4 Empirical Support

Three independent lines of evidence support the $O(\sqrt{p})$ claim:

1. Birthday collision timing. First collisions arrive at graph volume $\approx \sqrt{(\pi p/2)}$, matching the birthday bound for a uniform random walk on a space of size p . The ratio $V_{\text{first}}/\text{theory}$ was 1.063 and 0.689 in two runs (both within the birthday window $[0.5, 2.0]$). This is model-independent evidence — it does not depend on any matrix construction.

2. Full-Markov spectral gap. The true transition kernel has spectral gap 0.155, giving $O(1/\text{gap})/\sqrt{p} \approx 0.025$. This is not merely $O(1)$ but roughly $1/40$ of $\sqrt{p}$, meaning the walk is concretely fast, not just theoretically bounded.

3. Zero dead ends across all runs. The walk never fragments or terminates early. The graph is well-connected at every observed scale.

4.5 What Is Not Yet Proven

The argument is empirically very strong but not yet a theorem. What is missing is a proof that the isogeny graph on x-coordinates is an expander for all curves and all primes — i.e., a Weil-bound type estimate guaranteeing that $|\lambda_2|$ is bounded away from 1 uniformly. The empirical data is consistent with this but does not prove it. That is the open theoretical question.

5. Broader Mathematical Implications

5.1 Canonical Factor Base

Classical index calculus on $J(C)$ requires an ad hoc choice of factor base (e.g. divisors supported on small-norm points). Here the factor base emerges canonically from the fibration geometry: the d_1 atoms are exactly the x -coordinates of fiber intersection points, and the relations are principal divisors forced by Bezout. This is not a heuristic choice — the geometry dictates what the natural generators are. This is unusual and suggests the fibration reveals intrinsic structure of $J(C)$ not visible in the standard Mumford/Cantor presentation.

5.2 The Recurrence as Elliptic Group Law

The recurrence $x_k(m) = S(m) - 3x_i - x_j(m)$ is a Mobius-like transformation on the fiber. $S(m)$ is a degree-2 rational function, and the whole recurrence is exactly the elliptic curve addition law on the fiber E_m , projected back to the base. What the walk iterates is essentially repeated application of the elliptic group law, storing its complexity in the section polynomial. The mixing may be fast precisely because the group orders of the fibers $E_m(F_p)$ are incommensurate across different n -values, creating the entropy observed empirically.

5.3 Rational Points and Mordell-Weil Basis

The same walk can recover rational points on C over $\mathbb{Q}$. Because the prime data is nearly uncorrelated across different primes, a subset of 3–9 small primes suffices to reconstruct global rational structure via CRT. The local-to-global passage is dramatically cheaper than height-bound estimates suggest: the walk captures generator information for $J(C)(\mathbb{Q}) \bmod p$ with high efficiency, and those few constraints rapidly eliminate all but the correct global solution. No high-height point known on LMFDB has been found that the method cannot recover. If this holds generally, it is a practical algorithm for computing Mordell-Weil bases on genus-2 curves — currently a hard open problem in computational number theory.

5.4 The Auxiliary Curve Self-Similarity

The fact that $z^2 = B(m)$ is isomorphic to C itself — the curve appearing as its own auxiliary curve — is not a coincidence. This self-referential structure likely has a moduli-theoretic interpretation: the fibration may be an endomorphism of C (or a correspondence $C \rightarrow C$) inducing a specific endomorphism on $J(C)$. If so, the Markov walk implicitly computes in the endomorphism ring of $J(C)$, which would explain why it accesses structure invisible to generic index calculus.

5.5 Genus-3 and Higher Genus

The same approach has been tested on genus-3 curves from the MIT/LMFDB database. The prime data appears uncorrelated across primes in those cases as well, suggesting the method generalises. If so, the fibration walk represents a uniform attack on hyperelliptic cryptography at any

genus, reducing the effective security to the genus-1 level in each case.

6. Module Interaction Summary

The five modules interact as follows during a typical run:

genus2_markov_module.py

Entry point. Loads Sage sources, builds `search_fn` via `make_project_markov_search_fn()`, constructs `walker`, calls `walker.run()`, then drives post-hoc diagnostics.

walkerclass.py

Maintains walk state. Calls `search_fn` at each step, selects `x_j`, stores `RelationRecord`, feeds all three adjacency matrices via `_store_record()`, tracks leaves and collisions, prints per-step diagnostics.

adjacency_matrix.py

Receives one record per step via `ingest()`. Maintains three parallel sparse matrices (`chain` / `graph` / `full_markov`). Prints spectral gap reports periodically and at end of run.

mixing_diagnostics.py

Called once at end of run via `print_mixing_report()`. Also generates the `[mix]` one-liner printed after each step inside `walkerclass`.

relation_matrix.py

Called once at end of run. Encodes full history as integer divisor-relation matrix, computes rank mod Mersenne prime, reports shape and nullity.

Data flow

```
genus2_markov_module
→ builds search_fn (Mumford parallel search + Vieta enrichment)
→ constructs Genus2MetropolisWalker
→ walker.run(500)
for each step:
search_fn(x_i) → candidate pool (x_j, x_k, m, source, S_of_m)
Metropolis select x_j
_store_record(rec)
→ mat_chain.ingest(rec)
→ mat_graph.ingest(rec)
→ mat_full.ingest(rec)
mixing_one_liner() → [mix] line
mat_*.maybe_report() (cadenced)
end loop
```

```
mat_*.maybe_report(force=True) → spectral reports  
→ close_under_involution()  
→ cantor_summary()  
→ print_mixing_report() → birthday diagnostics  
→ build_relation_matrix2() → rank report  
→ walker.summary() → atom counts per matrix
```

7. Open Questions

Expander graph proof.

Prove that the isogeny graph on x-coordinates is an expander uniformly over all curves and primes. A Weil-bound type estimate for the spectral gap would convert the empirical $O(\sqrt{p})$ claim into a theorem.

Endomorphism ring interpretation.

Formalise the connection between the self-similar auxiliary curve and the endomorphism ring of $J(C)$. Does the walk compute in $\text{End}(J(C))$?

Full Mordell-Weil coverage.

Prove (or disprove) that the 3-9 prime CRT reconstruction always recovers all rational points, not just those accessible via the fibration section. What is the precise characterisation of which points the section can see?

Higher genus.

Extend the analysis to genus 3 and beyond. The genus-3 empirical tests are promising; a systematic study of the mixing constant as a function of genus and p would be valuable.

Subexponential improvement.

Does the structured entropy in $S(m)$ (degree-2 rational function) allow a subexponential improvement over the $O(\sqrt{p})$ birthday bound? Or is the birthday bound tight?

Kummer surface bias.

The theory.txt notes an intention to use the Kummer surface for a Metropolis bias on x_j selection. Would this improve the mixing constant or the birthday collision timing?